ASSIGNMENT 1
TASK 1 REPORT
Binary Classification using K-Nearest Neighbors (KNN)

Course: Large Language Models

KNN classifier Implemented from Scratch

Student Name	Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number	2621MA08, 2621MA12, 2621MA01
Group	08
Course	Large Language Models
Submission Date	16/08/2026

1. Introduction

In the task 1 of Assignment 1—binary classification using a K-Nearest Neighbors (KNN) classifier—is presented in this report. A CSV dataset with measurements taken from digital fine-needle aspirate images of breast masses is used in this task. Diagnosis is the target variable, with M standing for malignant and B for benign.

An 80% training and 20% testing split, KNN implementation from scratch, experimentation with K values 3, 4, 9, 20, and 47, comparison of Euclidean, Manhattan, Minkowski, Cosine Similarity, and Hamming distance measures, testing accuracy-based configuration selection, and evaluation using confusion matrix, precision, and recall are all required for this assignment. Additionally, a K-versus-accuracy plot is needed.

2. Objectives

- Get the dataset for breast cancer ready for classification.
- Distinguish the goal variable (diagnosis) from the predictor variables (features).
- Create a custom 80/20 train-test split with stratification.
- Standardize the input features using training-set statistics.
- Implement distance calculations required by KNN.
- Implement a KNN classifier from scratch without using a pre-built KNN model.
- Experiment with the specified K values and distance measures.
- Select the best-performing KNN configuration using test accuracy.
- Visualize the effect of K and distance metric on accuracy.

3. Dataset Description

The dataset contains 30 independent features describing characteristics of breast-cell nuclei, including mean, standard-error and worst-case measurements. The target variable is Diagnosis. The 30 feature names specified in the assignment include Radius_mean, Texture_mean, Perimeter_mean, Area_mean, Smoothness_mean, Compactness_mean, Concavity_mean, Concavepoints_mean, Symmetry_mean, Fractal_dimension_mean, corresponding standard-error features, and corresponding worst-case features.

The diagnosis label is encoded numerically for the classifier: M (malignant) is mapped to 1 and B (benign) is mapped to 0.

4. Data Preprocessing

4.1 Loading and cleaning

```
df = pd.read_csv('/data.csv')
df = df.drop(columns=['Unnamed: 32'], errors='ignore')
```

The CSV file is loaded into a pandas DataFrame. The Unnamed: 32 column is removed because it is an unnecessary column in the standard version of this dataset. The errors='ignore' option prevents failure if the column is already absent.

4.2 Encoding the target

```
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})
```

The categorical diagnosis labels are converted into binary numerical labels so that the classifier can perform numerical comparisons and voting.

4.3 Separating X and y

```
X = df.drop(columns=['id', 'diagnosis'])
y = df['diagnosis']
```

X contains the 30 predictive attributes. The ID column is excluded because it is only an identifier, and Diagnosis is excluded from X because it is the value to be predicted. y contains the target labels.

4.4 Train-test split

A custom train_test_split function was implemented. The split uses test_size=0.2, random_state=42 and stratify=y. Stratification is used so that the class proportions are approximately preserved in the training and testing subsets. The random seed makes the random selection reproducible.

```
X_train, X_test, y_train, y_test = custom_train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

4.5 Feature scaling

```
scaler = CustomStandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

A custom standard scaler was implemented. It computes the mean and standard deviation of each feature using the training data and applies the standardization formula (x - mean) / standard deviation. The scaler is fitted only on the training data and the learned statistics are then reused to transform the test data. This avoids using test-set information during preprocessing and therefore avoids data leakage.

5. Distance Measures

KNN classifies a sample by comparing it with stored training samples. The distance measure determines which training samples are considered nearest. The implementation contains the following distance functions:

- Euclidean distance: straight-line distance based on the squared differences of corresponding features.
- Manhattan distance: sum of the absolute differences of corresponding features.
- Minkowski distance: a generalized distance controlled by p; the current implementation uses p=3.
- Cosine distance: cosine similarity is converted to a distance using 1 - cosine similarity, so smaller values indicate greater similarity.
- Hamming distance: counts positions at which two vectors differ.

6. KNN Classifier Implementation

The KNNClassifier class stores K, the selected distance metric and the Minkowski p parameter. During fit(), KNN stores the training data and corresponding labels. For a test sample, the classifier computes distances to all training samples, sorts the distances, selects the K smallest distances and uses majority voting to select the predicted class.

```
class KNNClassifier:
    def __init__(self, k=3, distance_metric='euclidean', p_minkowski=3):
        self.k = k
        self.distance_metric = distance_metric
        self.p_minkowski = p_minkowski

    def fit(self, X, y):
        self.X_train = X
```

```
        self.y_train = y
```

The prediction procedure uses NumPy argsort to obtain the indices of the nearest neighbors and collections.Counter to count the class labels among those neighbors. The most frequent label is returned as the prediction.

7. Experimental Design

The model is evaluated for the K values specified by the assignment: 3, 4, 9, 20 and 47. The developed experiment loop currently tests Euclidean, Manhattan, Minkowski and Cosine Similarity. Each K-distance combination is fitted on the training data and evaluated on the test data. The K, metric and accuracy are stored in a results table and sorted by accuracy.

```
k_values = [3, 4, 9, 20, 47]
distance_metrics = ['euclidean', 'manhattan', 'minkowski', 'cosine_similarity']

results = []
for k in k_values:
    for metric in distance_metrics:
        model = KNNClassifier(k=k, distance_metric=metric, p_minkowski=3)
        model.fit(X_train_arr, y_train.values)
        accuracy = model.evaluate(X_test_arr, y_test.values)
        results.append({'K': k, 'Metric': metric, 'Accuracy': accuracy})
```

8. Model Selection

```
best_result = results_df.loc[results_df['Accuracy'].idxmax()]
```

The maximum test accuracy is identified using idxmax(). The corresponding row is retrieved with .loc, giving the best K, best distance metric and best testing accuracy.

9. Results

The numerical results below should be filled directly from the final Google Colab output. No accuracy value is assumed in this report.

Testing K=3, Metric=euclidean

Testing K=3, Metric=manhattan

Testing K=3, Metric=minkowski

Testing K=3, Metric=cosine_similarity

Testing K=4, Metric=euclidean

Testing K=4, Metric=manhattan

Testing K=4, Metric=minkowski

Testing K=4, Metric=cosine_similarity

Testing K=9, Metric=euclidean

Testing K=9, Metric=manhattan

Testing K=9, Metric=minkowski

Testing K=9, Metric=cosine_similarity

Testing K=20, Metric=euclidean

Testing K=20, Metric=manhattan

Testing K=20, Metric=minkowski

Testing K=20, Metric=cosine_similarity

Testing K=47, Metric=euclidean

Testing K=47, Metric=manhattan

Testing K=47, Metric=minkowski

Testing K=47, Metric=cosine_similarity

	K	**Metric**	**Accuracy**
14	*20*	*minkowski*	*0.955752*
9	*9*	*manhattan*	*0.955752*
0	*3*	*euclidean*	*0.946903*
1	*3*	*manhattan*	*0.946903*
18	*47*	*minkowski*	*0.946903*
19	*47*	*cosine_similarity*	*0.946903*
8	*9*	*euclidean*	*0.946903*
4	*4*	*euclidean*	*0.946903*
10	*9*	*minkowski*	*0.946903*
15	*20*	*cosine_similarity*	*0.946903*
12	*20*	*euclidean*	*0.946903*
6	*4*	*minkowski*	*0.946903*
17	*47*	*manhattan*	*0.938053*
2	*3*	*minkowski*	*0.938053*
5	*4*	*manhattan*	*0.938053*
13	*20*	*manhattan*	*0.938053*
16	*47*	*euclidean*	*0.938053*

	K	Metric	Accuracy
11	9	cosine_similarity	0.929204
7	4	cosine_similarity	0.929204
3	3	cosine_similarity	0.920354

9.1 Best configuration

Best K: 3

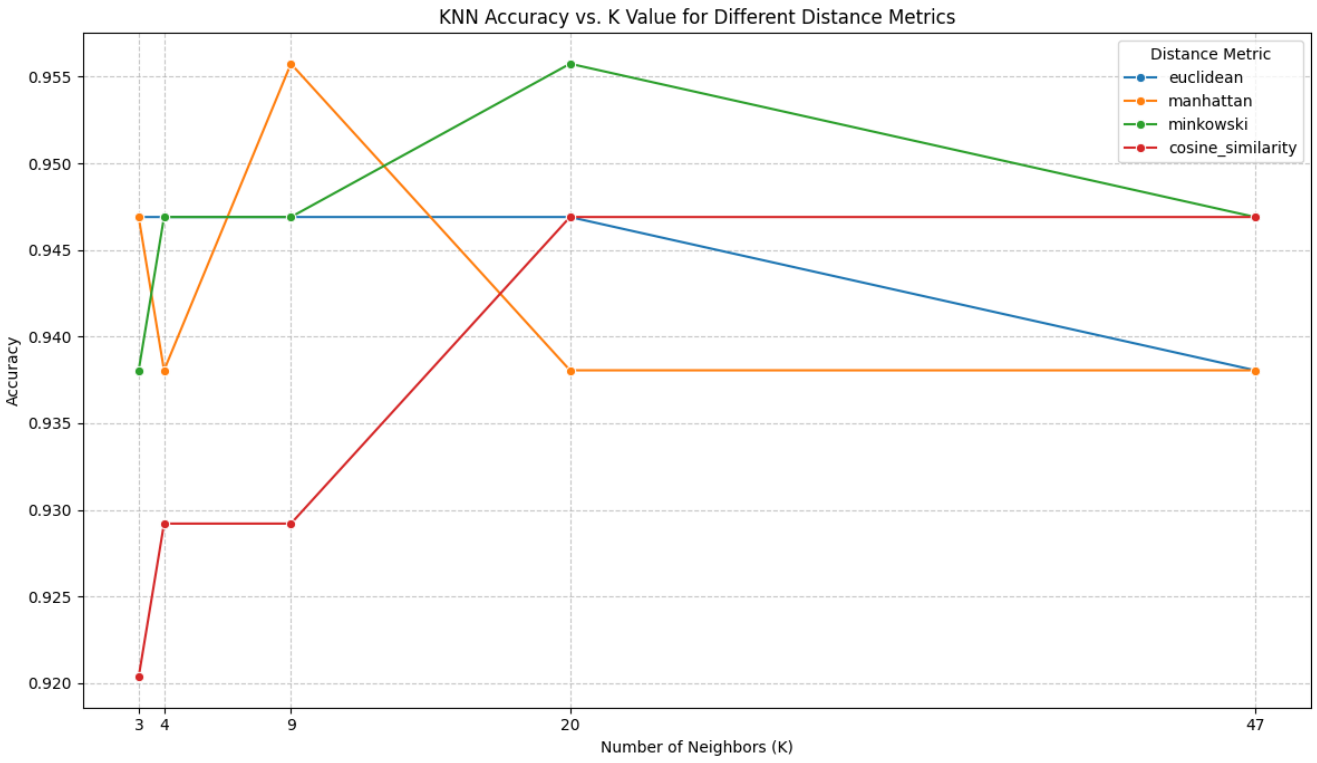
Best distance metric: manhattan

Best test accuracy: 0.9558

10. Required Plot

The assignment requires a graph of K values versus testing accuracy for the different distance metrics. The final report should include the plot generated from the completed results table and a short interpretation of which metric is strongest across K values and whether accuracy changes consistently as K increases.

Figure placeholder:



13. Limitations and Implementation Notes

There is an important implementation gap between the assignment specification and the Task 1 code currently developed: the assignment explicitly lists Hamming distance, but the current Task 1 experiment loop tests only Euclidean, Manhattan, Minkowski and Cosine Similarity, and the KNNClassifier does not yet contain a Hamming branch. Therefore the Task 1 implementation should be updated or the Hamming methodology should be confirmed with the instructor before submission.

The current Minkowski implementation uses p=3. The assignment specifies Minkowski distance but does not state a required p value in the supplied document. The chosen p=3 should therefore be stated explicitly in the final report and code.

14. Conclusion

Task 1 implements a binary KNN classification pipeline from data preparation through model selection. The dataset is cleaned, the diagnosis labels are encoded, the data are split into training and testing subsets using stratification, and the features are standardized using training-set statistics. A custom KNN classifier then uses multiple distance measures and majority voting to predict the diagnosis of unseen samples. The final model should be selected from the completed experiment table using the highest test accuracy, followed by confusion-matrix, precision and recall evaluation.

Before submission, the final Colab outputs should be copied into the Results section, the required K-versus-Accuracy plot should be inserted, and the Hamming-distance requirement should be resolved so that the implementation fully matches the assignment specification.